Mercadito: Implementación de la Primera Fase de una Plataforma Web de Compras en Mercados Locales

Sergio Luis Valdéz Pérez
Carné: 202111468
Universidad de San Carlos de Guatemala
Facultad de Ingeniería

Maria Fernanda Álvarez Morales
Carné: 202001537
Universidad de San Carlos de Guatemala
Facultad de Ingeniería

Introducción a la Programación de Computadoras [0980] — Primera fase, segundo semestre de 2026

***Resumen*—Se presenta el diseño, desarrollo y evaluación de la primera fase de *Mercadito*, una aplicación web orientada a consultar productos de mercados locales y registrar compras con entrega a domicilio en zonas seleccionadas de Guatemala y Mixco. El sistema emplea Python y Django para implementar navegación, autenticación, formularios, operaciones del carrito y confirmación de pedidos; PostgreSQL actúa como almacenamiento persistente. La solución distingue precios y tarifas por mercado, calcula los importes mediante aritmética decimal, registra precios históricos y comprueba la validez de las operaciones antes de guardar una compra. Se integraron controles de acceso, protección frente a confirmaciones duplicadas y transacciones para evitar pedidos incompletos. La verificación disponible corresponde a 88 pruebas automatizadas descubiertas: 86 concluyeron correctamente y dos fueron omitidas porque requieren concurrencia con PostgreSQL real. Los resultados respaldan el funcionamiento de los módulos implementados en el entorno probado, sin constituir una validación de despliegue en producción.**

***Index Terms*—Django, Python, PostgreSQL, comercio electrónico, programación orientada a objetos, carrito de compras, validación de datos.**

I. Introducción

La compra presencial de alimentos y artículos de consumo cotidiano requiere tiempo de traslado, búsqueda de productos y comparación de precios. Como alternativa académica, el proyecto *Mercadito* propone conectar a compradores con mercados cercanos mediante una plataforma web que permite seleccionar una zona, visualizar mercados con cobertura, explorar productos y confirmar pedidos para entrega en el domicilio [1], [2]. La referencia funcional de la propuesta es el proceso de compra en línea de *CENMA, Tu Mercado a un Clic*; esta referencia no implica que se hayan reutilizado sus componentes de software.

La primera fase se desarrolló con herramientas de programación web y persistencia relacional. El trabajo comprende tanto la interfaz gráfica como la definición de clases, métodos, instanciación de objetos, validación de entradas y almacenamiento de información, criterios contemplados en la rúbrica de evaluación del curso [3]. El presente reporte describe la implementación final proporcionada, no únicamente la propuesta inicial.

II. Problema, objetivos y alcance

II-A. Problema identificado

Los consumidores necesitan saber qué mercados atienden su ubicación, cuáles productos están disponibles y cuánto representa el costo total de la compra, incluido el envío. Sin un mecanismo unificado, dicha información se consulta de forma fragmentada y dificulta planificar pedidos a domicilio. Para los mercados, una interfaz de catálogo constituye un canal adicional de contacto con los compradores.

II-B. Objetivo general y objetivos específicos

Objetivo general: desarrollar una aplicación web operativa que permita gestionar la selección y compra de productos de mercados locales, con almacenamiento persistente en PostgreSQL.

Se definieron como objetivos específicos: a) consultar zonas, mercados y catálogos; b) registrar y autenticar usuarios; c) administrar cantidades y productos en carritos independientes por mercado; d) calcular subtotales, tarifa de entrega y total; e) confirmar pedidos con dirección y fecha válidas; y f) conservar el historial de las transacciones y responder de forma controlada ante entradas incorrectas [4], [2].

II-C. Delimitación funcional

El conjunto inicial de datos contiene dos mercados: *El Guarda* y *La Florida*. Sus coberturas se definen mediante zonas de Guatemala y Mixco, y cada relación mercado–zona establece una tarifa fija de envío. La Fig. 1 ilustra la selección de zona y la consulta de los mercados correspondientes.

Los productos se organizan en frutas, verduras, carnes, granos y abarrotes, con precios propios de cada mercado. La compra se paga **contra entrega**; esta fase no incorpora pasarela de tarjetas, inventario por existencias, geolocalización automática ni optimización de recorridos [4].

III. Arquitectura y herramientas

III-A. Tecnologías

La aplicación utiliza **Python 3**, **Django 5.2.18**, **PostgreSQL**, el conector `psycopg` 3.3.6 y `python-dotenv`, junto con HTML, CSS y Git/GitHub. Las versiones se identificaron en `requirements.txt` de la versión del



(a) Elección de zona

(b) Mercados y cobertura

Figura 1. Evidencias de la interfaz: selección de zona y mercados habilitados para la entrega. Capturas del manual de usuario suministrado.

Tabla I
REGISTROS DEL CONJUNTO INICIAL DE DEMOSTRACIÓN

Entidad	**Registros**
Zonas de entrega	10
Mercados	2
Relaciones mercado–zona	11
Categorías de productos	5
Productos	30
Ofertas de producto por mercado	60
Total de registros iniciales	118

Tabla II
ENTIDADES DEFINIDAS EN LA APLICACIÓN

`Modelo`	**Responsabilidad**
`Zona`	Municipio y número de zona.
`Mercado`	Establecimiento y estado de atención.
`MercadoZona`	Cobertura y tarifa de envío.
`Categoria`	Clasificación de productos.
`Producto`	Producto y unidad de venta.
`ProductoMercado`	Precio y disponibilidad en cada mercado.
`Direccion`	Dirección asociada a un usuario.
`Carrito`	Carrito de usuario y mercado.
`ItemCarrito`	Cantidad seleccionada por producto.
`Pedido`	Compra confirmada, domicilio, fecha e importes.
`ItemPedido`	Cantidad y precio histórico de cada producto.

proyecto entregada [5]. La conexión a la base de datos obtiene sus parámetros del archivo `.env`, mientras que `.env.example` sirve como referencia para cada instalación.

III-B. Distribución del código

Las rutas de `tienda/urls.py` conducen las solicitudes a `views.py`. Los formularios de `forms.py` recopilan y validan entradas; `validaciones.py` concentra las reglas de importes y cantidades; `servicios.py` coordina modificaciones del carrito y la compra; los modelos de `models.py` representan las entidades persistidas mediante el ORM de Django. Las plantillas y los estilos construyen las pantallas. La Fig. 2 representa esta separación de responsabilidades [5], [4].

III-C. Instalación y reproducción

Para una instalación nueva se clona el repositorio, se crea un entorno virtual de Python, se instalan las dependencias y se configura un usuario y una base de datos PostgreSQL. Posteriormente se prepara `.env`, se ejecutan las migraciones, se carga el *fixture* inicial y se crea un superusuario. Para instalaciones que ya disponen de datos, `loaddata` no debe repetirse porque podría sobrescribir registros editados [4].

IV. MODELO DE DATOS Y PROGRAMACIÓN ORIENTADA A OBJETOS

IV-A. Entidades y relaciones

Las entidades de `models.py` son modelos de Django que definen atributos, relaciones y operaciones. La Tabla II resume las once entidades del dominio, además de utilizar el modelo integrado de usuario de Django. `MercadoZona` y `ProductoMercado` modelan respectivamente la cobertura y la oferta particular de cada mercado.

Cada `Carrito` se vincula con un usuario y un mercado, y no admite dos registros con esa misma combinación. De forma semejante, un producto no puede aparecer dos veces en el mismo carrito: cuando se agrega nuevamente, la cantidad se acumula. Una compra genera un `Pedido` y sus `ItemPedido`. Estos últimos guardan el precio al momento de confirmar, de modo que el importe de una compra antigua no depende de ediciones posteriores del catálogo.

IV-B. Métodos implementados y cálculos

Además de `__str__()`, destacan `obtener_cobertura()` en Mercado, `calcular_importe()` en ProductoMercado y los métodos `detalle_items()`, `calcular_subtotal()`, `calcular_envio()` y `calcular_total()` en Carrito. La clase Pedido define `actualizar_totales()` [5], [4]. Los objetos se instancian al crear y modificar compras.

El sistema utiliza `Decimal`, no el tipo binario `float`, y

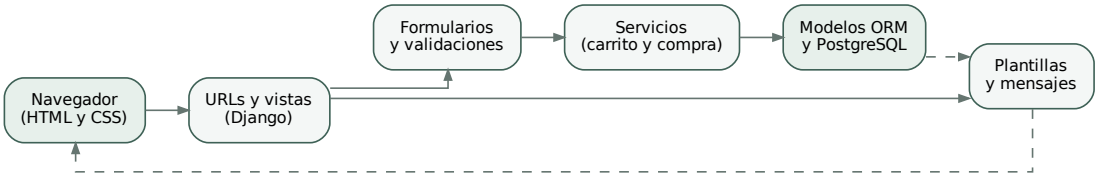


Figura 2. Arquitectura lógica implementada, sintetizada a partir de las aplicaciones y archivos del repositorio. Las flechas discontinuas representan el retorno de información hacia la interfaz.

redondea cada importe a centavos:

$$S_i = \text{redondear}_{0,01}(q_i p_i), \tag{1}$$

$$S = \sum_{i=1}^{n} S_i, \tag{2}$$

$$T = S + E(m, z), \tag{3}$$

siendo q_i la cantidad del producto i, p_i su precio en el mercado, $E(m, z)$ la tarifa para el mercado m y zona z, y T el total. El Listado 1 presenta el método utilizado para normalizar importes.

```
def dinero(valor):
    """Redondea cada importe a centavos, sin convertirlo a
     float."""
    return Decimal(valor).quantize(CENTAVO, rounding=
     ROUND_HALF_UP)
```

Listing 1. Método de redondeo utilizado en validaciones.py (extracto literal).

Como comprobación aritmética del catálogo inicial, tres libras de manzana a Q8.00 por libra en El Guarda representan Q24.00. Para la zona 3 de Guatemala la tarifa de Q15.00 produce un total de **Q39.00**. Este es un caso de demostración registrado en los datos del proyecto, no un precio comercial actualizado.

V. Implementación de funciones

V-A. *Consulta, registro y autenticación*

La pantalla inicial permite elegir zona y consultar mercados y productos aun sin estar autenticado. Para comprar, `RegistroForm` recoge nombre de usuario, nombres, apellidos, correo y contraseña repetida; `IngresoForm` emplea el mecanismo de autenticación de Django. El cierre de sesión no elimina los carritos ni los pedidos guardados. La Fig. 3 evidencia el catálogo y el formulario de creación de cuenta.

V-B. *Gestión de carritos*

El servicio `agregar_producto()` verifica la cobertura, la disponibilidad y la cantidad; recupera o crea el carrito del usuario para un mercado. Los servicios `actualizar_item()`, `quitar_item()` y `vaciar_carrito()` registran los cambios en la base. El sistema muestra cantidades, precios actuales, subtotales, tarifa de entrega y total; las operaciones de un mercado no mezclan artículos de otro [2], [4].

Las libras y los litros admiten fracciones hasta dos decimales; las unidades y docenas requieren cantidades enteras. El mínimo permitido es 0,01 para productos fraccionables o una unidad completa para los demás, y el máximo es 9999,99 o 9999, respectivamente.

V-C. *Confirmación del pedido e historial*

El usuario introduce una dirección entre 8 y 300 caracteres, después de normalizar espacios, y selecciona una fecha igual o posterior al día actual. El formulario presenta el resumen calculado en el servidor: ningún subtotal o precio enviado desde el navegador se utiliza como autoridad. Al confirmar, se guardan el pedido y sus renglones con el precio vigente, se vacía el carrito y se presenta el detalle. El pago se mantiene contra entrega.

El módulo *Mis pedidos* ofrece historial paginado y detalle de cada transacción, con estado, fecha y montos. Las consultas se limitan al usuario autenticado, de manera que no pueden visualizarse pedidos ajenos. La Fig. 5 muestra las pantallas finales.

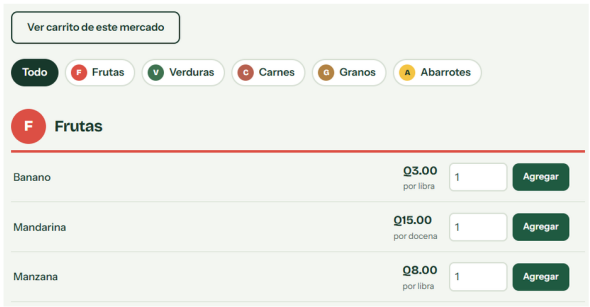
VI. Controles de calidad y persistencia

VI-A. *Validaciones y excepciones*

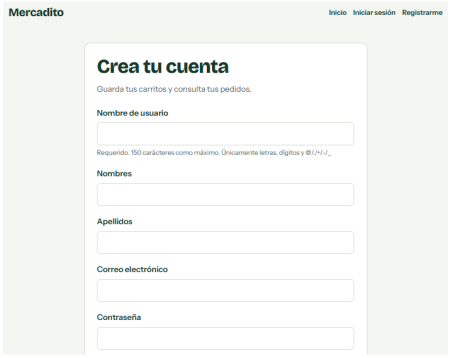
La interfaz informa sobre datos obligatorios, credenciales erróneas, cantidades inválidas, mercados sin cobertura, ofertas retiradas y fechas anteriores al día de compra. Las vistas que modifican registros utilizan solicitudes `POST` con protección CSRF; los carritos y pedidos requieren autenticación. Los formularios conservan los datos útiles cuando fallan, muestran avisos y redirigen después de completar una operación para evitar reenvíos accidentales [4].

VI-B. *Confirmación segura y transacción*

Para confirmar una compra se genera un token firmado vinculado al usuario, carrito y una huella del resumen; su vigencia es de dos horas. Durante la operación, el servidor vuelve a consultar la cobertura, los artículos, las cantidades,

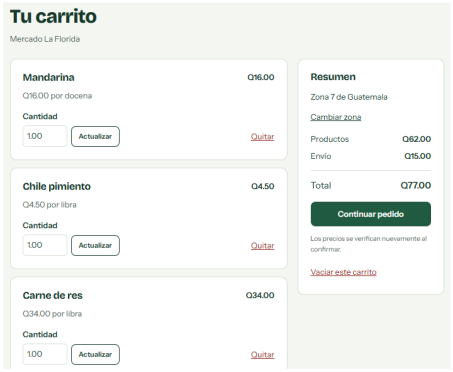


(a) Catálogo y unidades



(b) Registro de usuario

Figura 3. Catálogo por categorías y formulario de registro de usuario en la versión final. Capturas proporcionadas en el manual de usuario.



(a) Carrito y totales

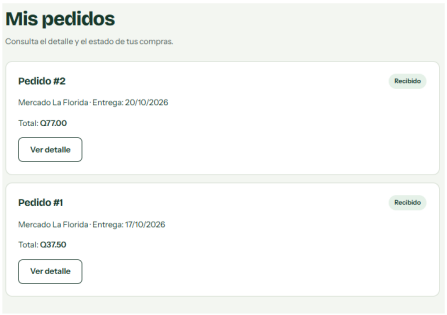


(b) Confirmación del pedido

Figura 4. Administración del carrito y formulario de confirmación con importes y entrega. Capturas del manual de usuario.



(a) Detalle de la compra



(b) Historial personal

Figura 5. Detalle de un pedido confirmado e historial personal de compras. Capturas del manual de usuario.

los precios y la tarifa. Si existe un cambio, solicita revisar el resumen en vez de registrar importes desactualizados.

La función `confirmar_compra()` está protegida por `transaction.atomic()`: crea el `Pedido`, añade los `ItemPedido`, actualiza los importes y vacía el carrito en una sola transacción. Si ocurre una excepción, se revierten los cambios. Un identificador de confirmación único permite reconocer un reenvío y devolver el mismo pedido sin duplicarlo. El código utiliza bloqueo de registros con `select_for_update()` para las operaciones concurrentes sobre el carrito [5]. Esas garantías deben seguir verificándose bajo concurrencia en un servidor PostgreSQL real.

VI-C. Evolución del esquema y consulta

La migración `0002_completar_compra.py` incorpora el token único de confirmación y restricciones sobre cantidades, importes y consistencia del total. El ORM lee y

Tabla III
Comprobaciones representativas de la primera fase

Área	Comportamiento comprobado
Autenticación	Registro, ingreso, salida y protección de operaciones privadas.
Carrito	Agregar, acumular, editar, quitar y vaciar productos; cantidades y cobertura.
Cálculo	Precios por mercado, redondeo decimal, subtotal, envío y total.
Pedido	Fecha, dirección, conservación de precios, confirmación repetida y reversión ante errores.
Interfaz	Renderizado de registro, catálogo, carrito, confirmación y consultas de pedidos.

escribe los datos en PostgreSQL en el modo habitual de ejecución. En pgAdmin pueden inspeccionarse, entre otras, las tablas `auth_user`, `tienda_carrito`, `tienda_pedido` y `tienda_itempedido`. El Listado 2 ilustra consultas no destructivas utilizadas para revisión.

```
SELECT * FROM auth_user LIMIT 20;
SELECT * FROM tienda_carrito LIMIT 20;
SELECT * FROM tienda_pedido ORDER BY creado DESC LIMIT 20;
```

Listing 2. Consultas de verificación de datos almacenados.

VII. Algoritmo y diagramas de flujo

El algoritmo íntegro de 26 pasos entregado por el equipo se conserva en el **Apéndice A**. Sus bloques principales son: selección y validación de zona; consulta de mercados y catálogo; registro e inicio de sesión; gestión del carrito; revisión de importes; validación de domicilio, fecha y token; registro transaccional de compra; y consulta del historial. Los dos diagramas suministrados se incluyen en el **Apéndice B**: el primero cubre navegación, sesión y carrito; el segundo describe la confirmación y persistencia de pedidos [6].

VIII. Resultados y comprobaciones

VIII-A. Cobertura funcional observada

Las evidencias gráficas del manual de usuario documentan el recorrido desde la elección de zona hasta el detalle del pedido. El conjunto de demostración contiene 118 registros iniciales y muestra la relación entre cobertura, ofertas, usuarios, carrito y compras. Se validaron los escenarios representados en la Tabla III con la suite de pruebas del proyecto [5], [2].

VIII-B. Resultados de la suite automatizada

En el entorno de prueba documentado se descubrieron **88 pruebas**: **86 finalizaron correctamente y 2 se omitieron**, sin fallos registrados. La ejecución se hizo con **Python 3.13, Django 5.2.18 y SQLite en memoria**; las pruebas omitidas requieren PostgreSQL real y evalúan adiciones y confirmaciones simultáneas. Una simulación aislada también verificó que la migración de la versión inicial conservaba datos previos. La revisión estática de la interfaz se realizó a 1280, 390 y 320 píxeles, sin desbordamiento horizontal observado en las combinaciones revisadas [7].

Por tanto, los resultados **no equivalen** a una prueba de carga, a una auditoría de seguridad o a una validación integral de concurrencia y migraciones en el PostgreSQL del usuario. Los manuales describen los comandos necesarios para ejecutar esas verificaciones en la instalación local.

VIII-C. Correspondencia con la rúbrica

El proyecto documenta propuesta y bienvenida funcional; clases, atributos, métodos e instanciación; entradas y navegación; lectura/escritura de datos; condiciones, excepciones y limpieza posterior a operaciones correctas. La documentación del equipo incluye el manual técnico, el manual de usuario, el algoritmo y los diagramas exigidos por la primera fase. Esta correspondencia es una revisión de **requisitos implementados**, no una calificación docente [3].

IX. Discusión y limitaciones

El proyecto resuelve el flujo de compra propuesto dentro de un alcance de demostración, pero no contiene pagos electrónicos, inventario por existencias, asignación logística ni ubicación automática. La oferta de productos se representa mediante un indicador de disponibilidad, por lo que no se garantiza control de existencias. La configuración de desarrollo contempla `DEBUG=True`; en consecuencia, el sistema no debe describirse como listo para exposición pública sin endurecimiento adicional de seguridad, despliegue y operación [4].

X. Conclusiones

1. Se integró un flujo funcional de consulta de mercados, autenticación, carrito, confirmación e historial de pedidos, manteniendo la diferenciación de productos y tarifas por mercado.
2. La estructura de modelos y servicios facilita ejemplificar programación orientada a objetos, relaciones de base de datos y cálculo decimal de importes con persistencia de precios históricos.
3. Las validaciones de servidor, las transacciones y el token de confirmación cubren casos relevantes de datos erróneos, escritura parcial y duplicación de pedidos. La suite de pruebas respalda estas funciones en el entorno documentado.
4. Como siguiente etapa, corresponde verificar la concurrencia y las migraciones sobre PostgreSQL real y abordar las limitaciones necesarias para un despliegue fuera del laboratorio.

Repositorio del proyecto

El código fuente colaborativo de Mercadito está disponible para los integrantes autorizados en https://github.com/Mafrezz/Mercadito. Los archivos complementarios de esta entrega incluyen los manuales, el algoritmo y los diagramas originales aportados por el equipo.

Apéndice A
Algoritmo completo de la primera fase

Los 26 pasos siguientes sintetizan el procedimiento entregado por el equipo y contemplan la validación de datos antes de la persistencia.

1. Abrir Mercadito; consultar zonas y categorías en PostgreSQL y presentar la bienvenida.
2. Solicitar zona de entrega; si es inválida o inexistente, informar y repetir la selección.
3. Guardar la zona en sesión y buscar mercados activos con cobertura y tarifa; si no hay, regresar a la selección.
4. Ordenar los mercados por costo de envío y solicitar la elección de uno.
5. Verificar que el mercado esté activo y cubra la zona; en caso contrario, informar y elegir de nuevo.
6. Consultar ofertas disponibles, agruparlas por categoría y mostrar el catálogo con filtros.
7. Al comenzar la compra, comprobar que el usuario haya iniciado sesión; en caso contrario, solicitar registro o ingreso.
8. En el registro validar los campos y la contraseña, crear la cuenta y dirigir al ingreso; autenticar credenciales.
9. Solicitar oferta y cantidad; validar mercado, disponibilidad y cobertura.
10. Comprobar cantidades positivas, límite máximo y uso de enteros para unidades y docenas.
11. Recuperar o crear el carrito del usuario para ese mercado; agregar producto o acumular su cantidad.
12. Mostrar el carrito y permitir actualizar cantidades, quitar renglones o vaciar; guardar cada modificación.
13. Calcular subtotales con precios actuales y sumar la tarifa fija de envío para obtener el total.
14. Impedir la confirmación si el carrito está vacío, falta cobertura o existe algún producto inválido.
15. Presentar el resumen correcto y producir un token firmado con la huella de los importes y artículos.
16. Solicitar dirección completa y fecha de entrega igual o posterior a la fecha presente.
17. Informar los errores de validación y conservar los datos aprovechables del formulario.
18. Verificar la firma y vigencia del token; si la confirmación ya se consumió, recuperar el pedido existente.
19. Volver a consultar cantidades, precios, estado de ofertas, zona y tarifa; comparar la huella del resumen.
20. Si hubo modificaciones, suspender la compra y solicitar revisar otra vez el resumen.
21. Iniciar una transacción y crear el pedido con usuario, mercado, zona, dirección, fecha y montos.
22. Crear los renglones `ItemPedido` con precios vigentes, actualizar totales y vaciar el carrito.
23. Si ocurre una excepción de escritura, revertir la transacción e informar al usuario.
24. Si la transacción concluye, presentar el pedido confirmado y conservarlo en *Mis pedidos*.
25. Al consultar el historial, recuperar solo compras del usuario autenticado y permitir ver detalle y estado.
26. Finalizar el proceso con pago contra entrega, sin procesamiento de cobros electrónicos.

Referencias

[1] Equipo Mercadito, *Propuesta inicial: plataforma web para compra y entrega de productos de mercado*, documento del proyecto, 2026.

[2] Equipo Mercadito, *Manual de usuario final*, documento proporcionado, oct. 2026.

[3] Curso [0980], *Rúbrica de Primera Fase: Introducción a la Programación de Computadoras*, material académico proporcionado, 2026.

[4] Equipo Mercadito, *Manual técnico final*, documento proporcionado, oct. 2026.

[5] Equipo Mercadito, *Mercadito: código Python/Django, modelos, formularios, servicios y migraciones de la primera fase*, paquete de código actualizado suministrado, oct. 2026.

[6] Equipo Mercadito, *Algoritmo y diagrama de flujo final*, documento proporcionado, oct. 2026.

[7] Equipo Mercadito, *Pruebas y revisión de la actualización*, registros adjuntos al código del proyecto, oct. 2026.

Apéndice B

Diagramas de flujo originales de la primera fase

Los siguientes diagramas proceden del archivo *Algoritmo y diagrama de flujo final.pdf*, proporcionado por el equipo. Se incluyen en página completa para conservar legibilidad.

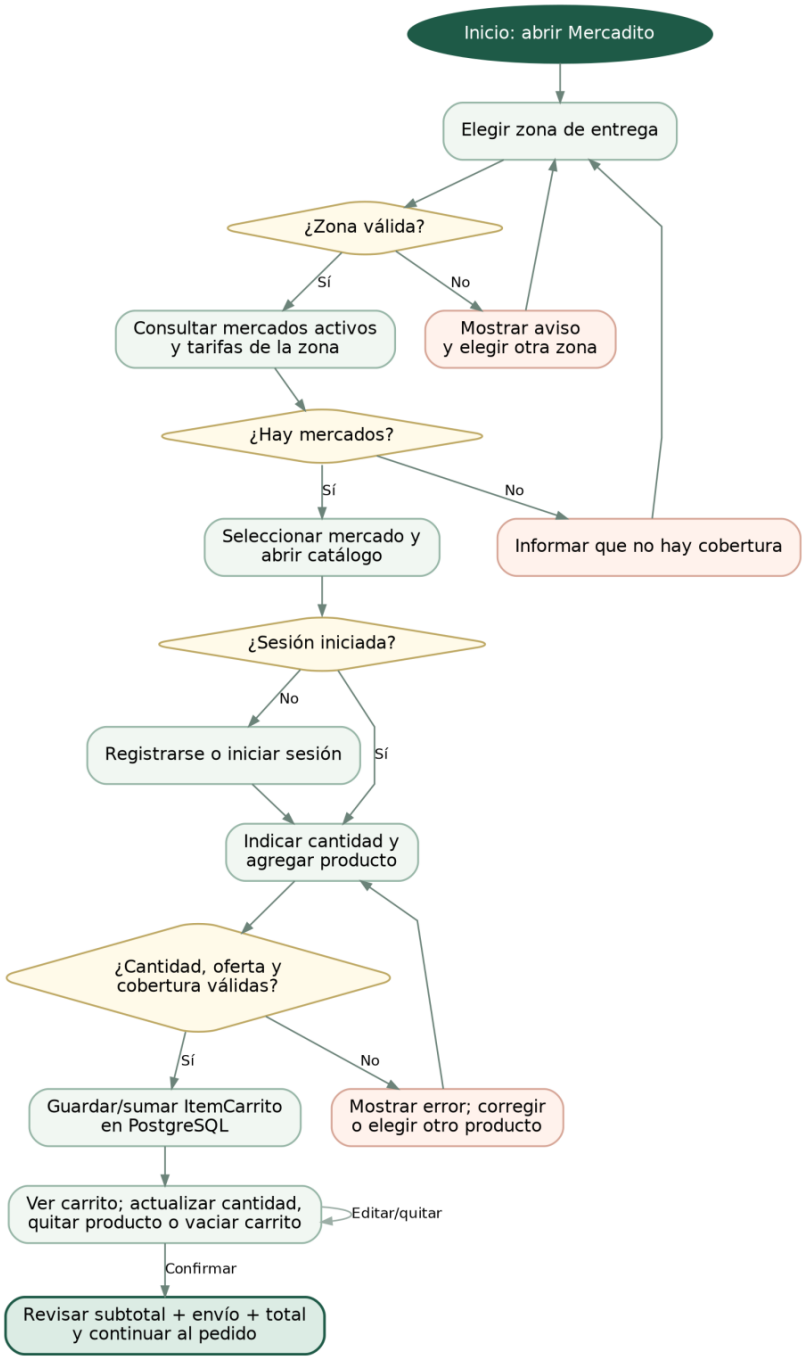


Figura 6. Selección de zona, autenticación y gestión del carrito. Diagrama suministrado por el equipo.

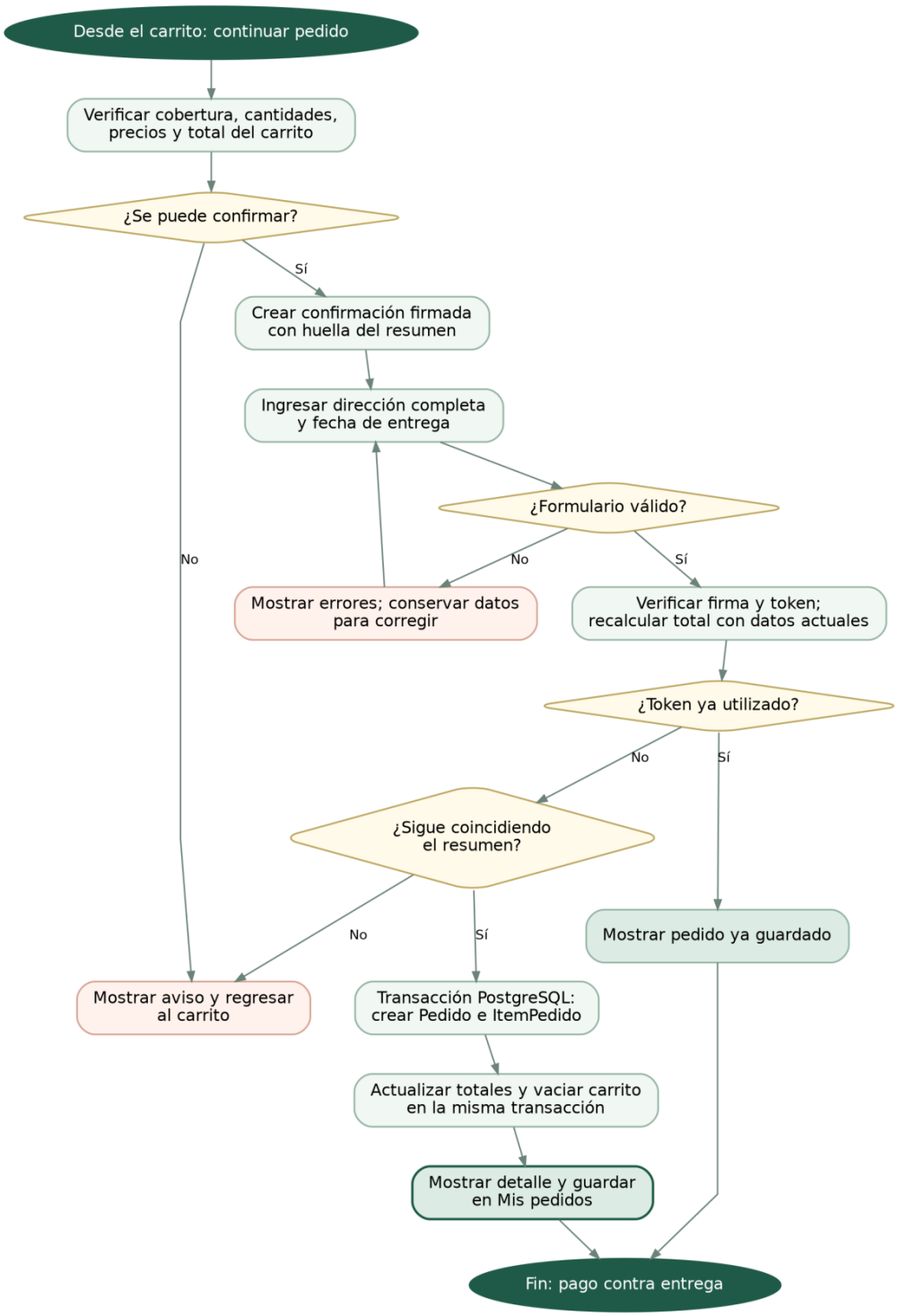


Figura 7. Confirmación, controles del resumen y persistencia transaccional del pedido. Diagrama suministrado por el equipo.